

Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought

<https://arxiv.org/pdf/2501.04682>

LLM reasoning has become quite a popular topic recently, due to emergence (and success) of such models as OpenAI's o1 and o3 and DeepSeek's R1. The o3 model became especially famous after making quite a significant progress in some of the very challenging benchmarks including [ARC AGI](#) and [Frontier math](#).

But what made them so successful? It's long been known that LLMs do their job better when allowed to produce a solution instead of just outputting the answer – quite likely because more compute is used for producing the final result (and not just because “LLM's think like humans” as you probably could have guessed). But o1 and o3 (and also experimental Gemini, DeepSeek-R1 and others) not only produce Chain of Thoughts solutions. Their reasoning process may be nonlinear, with iterations of reflection, self-correction, backtracking etc. And this, in turn, mirrors the human way of solving the problems: we often do some experimentation or take several preliminary attempts before writing down the correct solution.

With this in mind, it would be cool to train our own homebrew o3 for this nonlinear type of reasoning, but the problem is: where to get the data? (Without involving costly human creators, of course.) Collecting even just a good CoT dataset is a challenge – how on the Earth are we going to get the nonlinear reasoning dataset? Even math books can't help us with that: of course, they contain a number of proofs, but without backtracking an experimentation.

The authors of this paper suggest an interesting way of doing this. Let's assume that we're dealing with math problems. The benefit is that their solutions can be divided into logical steps and an LLM may be prompted to actually generate one step at a time. Now, how would we generate a valid solution? For that, we'd want to score an intermediate solutions ($s_1 \dots s_i$, up to step s_i). The most straightforward way would be to:

- Starting from $s_1 \dots s_i$, generate N continuations and define the score as the probability of arriving at the correct answer. Let's call this “**terminal-guided scoring**” (I've borrowed this term from the next paper))

Note that this scores the whole $s_1 \dots s_i$, not just s_i .

Now, having the correct answer and this scoring procedure, we can use [MCTS \(Monte Carlo Tree Search\)](#) to construct the optimal (and hopefully correct) solution.

When the tree is ready, we could just take the optimal solution; but instead, **let's gather all the nodes into one long solution, which will not contain both failed experiments and backtracking.**

This is, of course, very computationally intensive due to terminal-guided scoring and a generous amount of simulations (the authors talk about \$100 per tree). So, the authors adopt instead a somewhat simpler approach related to the A* algorithm.

This data may now be used for instruction tuning of an LLM; but the authors also propose the subsequent RL training phase to make solutions even better. I don't want to get into RL details. The authors actually leverage some good ways of dealing with latent variable models and arrive at the following loss

$$\begin{aligned} & \max_{\theta} \mathbb{E}_{\mathbf{Z} \sim \pi_{\theta}(\cdot|\mathbf{q}), \mathbf{S}, \mathbf{q} \sim \mathcal{D}_{\text{train}}} [\mathbf{sg}(\log \pi_{\theta}(\mathbf{S}|\mathbf{Z}, \mathbf{q})) - \beta \mathbb{D}_{KL}[\pi_{\theta}(\mathbf{Z}, |\mathbf{q})||\pi_{\text{ref}}(\mathbf{Z}|\mathbf{q})]] + \\ & \max_{\theta} \mathbb{E}_{\mathbf{Z} \sim \mathbf{sg}(\pi_{\theta}(\cdot|\mathbf{q})), \mathbf{S}, \mathbf{q} \sim \mathcal{D}_{\text{train}}} [\log \pi_{\theta}(\mathbf{S}|\mathbf{Z}, \mathbf{q})] \end{aligned} \quad (20)$$

where π_{θ} is the LLM, $\mathbf{q}$ is the query (the math problem itself), $\mathbf{Z}$ are the intermediate experiments and $\mathbf{S}$ is the final solution; also $\mathbf{sg}$ stands for stop gradients (which means that whatever is inside isn't subject to error backpropagation). The cool thing here is that the final solution $\mathbf{S}$ needn't be generated by the LLM; both the problem $\mathbf{q}$ and the solution $\mathbf{S}$ are taken from the dataset, and only $\mathbf{Z}$, the intermediate attempts at solution, are actually generated.

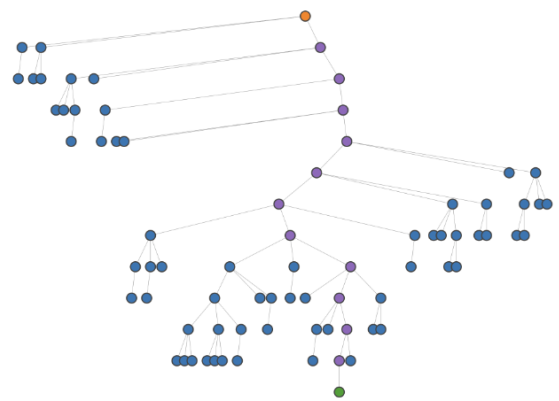


Figure 12: MCTS tree for the math problem presented by OpenAI (2024). The red node indicates the solution.

rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking

<https://arxiv.org/pdf/2501.04519>

This paper is also about reasoning :) It authors take a challenge of training a relatively small model (such as Qwen2-Math-7B or Phi3-mini) to solve math tasks as well as OpenAI's o1-preview (with the help of a larger LLM with 256B parameters as a training data generator, which somewhat contradicts the authors' claim that they don't distill from larger models).

rStar-Math approach is also based on step-by-step generation and MCTS, but there are some interesting new ideas.

First of all, the authors add one more layer of solution verification, at least for math problems. For that, they teach the LLM to produce solution as Python code with textual comments as code comments. Now, if the code executes successfully, we can hope that the overall math reasoning is correct.

Question: Bill walks $\frac{1}{2}$ mile south, then $\frac{3}{4}$ mile east, and finally $\frac{1}{2}$ mile south. How many miles is he, in a direct line, from his starting point? Express your answer as a decimal to the nearest hundredth.

Python code execution for step 1:

```
# Step 1: Calculate the total distance walked south
total_south = 1/2 + 1/2
# Step 2: Calculate the total distance walked east
total_east = 3/4
# Step 3: Use the Pythagorean theorem to find the direct distance from the starting point
import math
direct_distance = math.sqrt(total_south**2 + total_east**2)
# Step 4: Round the direct distance to the nearest hundredth
direct_distance_rounded = round(direct_distance, 2)
From the result, we can see that the direct distance from the starting point is \boxed{1.25} miles
```

Python code execution for step 2:

```
# Step 1: Calculate the total distance walked south
total_south = 1/2 + 1/2
# Step 2: Calculate the total distance walked east
total_east = 3/4
```

Of course, this particular approach to solution verification has serious restrictions, but it might really help to reduce hallucinations.

Next, instead of a **Process Reward Model (PRM)** which is trained to **predict** an intermediate solution's absolute score (which is estimated by Monte Carlo and this unreliable), the authors suggest using **Process Preference Model (PPM)** which is trained to **rank** intermediate solutions like a reward model in RLHF training would. To train PPM, the authors suggest taking:

- two correct trajectories with the highest average Q-values as positive examples and
- two incorrect trajectories with the lowest average Q-values as negative examples.

By Q-values we mean here the intermediate solution scores that we've somehow estimated, for example, using Monte Carlo.

The LLM and the PPM are trained alongside each other in a 4-step process:

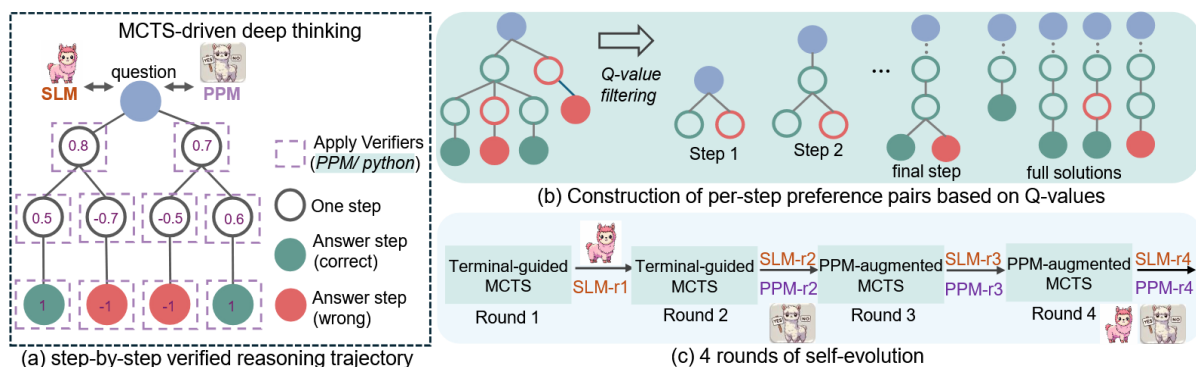


Figure 1: The overview of rStar-Math.

Round 1. A larger LLM (DeepSeek-Coder-V2-Instruct) is used to produce the first pack of training data. No PPM is available at this point, so they use MCTS, scoring an intermediate solution by generating 8 continuations at each step, evaluating final solutions by correctness of their answer and then backpropagating this back to the intermediate solutions (more or less, an intermediate solution gets a score ~ ratio of its continuations that lead to a correct answer). Training the small LLM on this data yields the model **SLM-r1**.

Round 2. On this stage, the first viable Process Preference Model, **PPM-r2**, is trained. For that, the authors calculate scores by running MCTS with SLM-r1 and 16 rollouts (which is more than 8 and gives a somewhat reliable PPM). MCTS is also used to create training data for fine tuning of SLM-r1 into **SLM-r2**.

Round 3. Now, PPM-r2 and SLM-r2 are used to create training data for both **PPM-r3** and **SLM-r3**.

Round 4. At this stage, some challenging problems from the training dataset are still not solved; to mitigate this, up to 128 next steps may be generated at each step. With the sheer power of chance, some of the hard problems may be eventually solved.

You can check the results in the large table below. They are quite promising.

Table 5: The results of rStar-Math and other frontier LLMs on the most challenging math benchmarks. rStar-Math⁶⁴ shows the Pass@1 accuracy achieved when sampling 64 trajectories.

Model	Method	Competition and College Level					OOD	
		MATH	AIME 2024	AMC 2023	Olympiad Bench	College Math	GSM8K	Gaokao En 2023
Frontier LLMs								
GPT-4o	System 1	76.6	9.3	47.5	43.3	48.5	92.9	67.5
Claude3.5-Sonnet	System 1	78.3	16.0	-	-	-	96.4	-
GPT-o1-preview	-	85.5	44.6	90.0	-	-	-	-
GPT-o1-mini	-	90.0	56.7	95.0	65.3	57.8	94.8	78.4
Open-Sourced Reasoning LLMs								
DeepSeek-Coder-V2-Instruct	System 1	75.3	13.3	57.5	37.6	46.2	94.9	64.7
Mathstral-7B-v0.1	System 1	57.8	0.0	37.5	21.5	33.7	84.9	46.0
NuminaMath-72B-CoT	System 1	64.0	3.3	70.0	32.6	39.7	90.8	58.4
LLaMA3.1-8B-Instruct	System 1	51.4	6.7	25.0	15.4	33.8	76.6	38.4
LLaMA3.1-70B-Instruct	System 1	65.4	23.3	50.0	27.7	42.5	94.1	54.0
Qwen2.5-Math-72B-Instruct	System 1	85.6	30.0	70.0	49.0	49.5	95.9	71.9
Qwen2.5-Math-72B-Instruct+72B ORM	System 2	85.8	36.7	72.5	54.5	50.6	96.4	76.9
General Base Model: Phi3-mini-Instruct (3.8B)								
Phi3-mini-Instruct (base model)	System 1	41.4	3.33	7.5	12.3	33.1	85.7	37.1
rStar-Math (3.8B SLM+7B PPM)	System 2	85.4	40.0	77.5	59.3	58.0	94.5	77.1
rStar-Math ⁶⁴ (3.8B SLM+7B PPM)	System 2	86.4	43.3	80.0	60.3	59.1	94.7	77.7
Math-Specialized Base Model: Qwen2.5-Math-1.5B								
Qwen2.5-Math-1.5B (base model)	System 1	51.2	0.0	22.5	16.7	38.4	74.6	46.5
Qwen2.5-Math-1.5B-Instruct	System 1	60.0	10.0	60.0	38.1	47.7	84.8	65.5
Qwen2.5-Math-1.5B-Instruct+72B ORM	System 2	83.4	20.0	72.5	47.3	50.2	94.1	73.0
rStar-Math (1.5B SLM+7B PPM)	System 2	87.8	46.7	80.0	63.5	59.0	94.3	77.7
rStar-Math ⁶⁴ (1.5B SLM+7B PPM)	System 2	88.6	46.7	85.0	64.6	59.3	94.8	79.5
Math-Specialized Base Model: Qwen2-Math-7B								
Qwen2-Math-7B (base model)	System 1	53.4	3.3	25.0	17.3	39.4	80.4	47.3
Qwen2-Math-7B-Instruct	System 1	73.2	13.3	62.5	38.2	45.9	89.9	62.1
Qwen2-Math-7B-Instruct+72B ORM	System 2	83.4	23.3	62.5	47.6	47.9	95.1	71.9
rStar-Math (7B SLM+7B PPM)	System 2	88.2	43.3	80.0	63.1	58.4	94.6	78.2
rStar-Math ⁶⁴ (7B SLM+7B PPM)	System 2	88.6	46.7	85.0	63.4	59.3	94.8	79.2
Math-Specialized Base Model: Qwen2.5-Math-7B								
Qwen2.5-Math-7B (base model)	System 1	58.8	0.0	22.5	21.8	41.6	91.6	51.7
Qwen2.5-Math-7B-Instruct	System 1	82.6	6.0	62.5	41.6	46.8	95.2	66.8
Qwen2.5-Math-7B-Instruct+72B ORM	System 2	88.4	26.7	75.0	49.9	49.6	97.9	75.1
rStar-Math (7B SLM+7B PPM)	System 2	89.4	50.0	87.5	65.3	59.0	95.0	80.5
rStar-Math ⁶⁴ (7B SLM+7B PPM)	System 2	90.0	53.3	87.5	65.6	60.5	95.2	81.3

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf

The long-awaited DeepSeek-R1 model [is finally available](#). It's highly competitive, even among the top LLMs from Open AI and Anthropic. And it's open source. And R hints at its reasoning capabilities, of course ;)

Now that we've seen some recent research around establishing reasoning capabilities, let's see how major players do this. There are actually two models: DeepSeek-R1-Zero and DeepSeek-R1, fine tuned from DeepSeek-V3-Base.

An interesting thing about DeepSeek-R1-Zero is that it skips SFT and starts from RL. RL operates with two types of reward:

- Accuracy rewards. The authors prompt the model to output answers in specific formats, when possible, to allow for deterministic answer extraction. For example, this is done in math tasks.
- Format following reward that enforces the model to put its thinking process between '<think>' and '</think>'.

Note that there are no process rewards, but quite curiously the LLM still learns not only to provide answer but also to produce solutions:

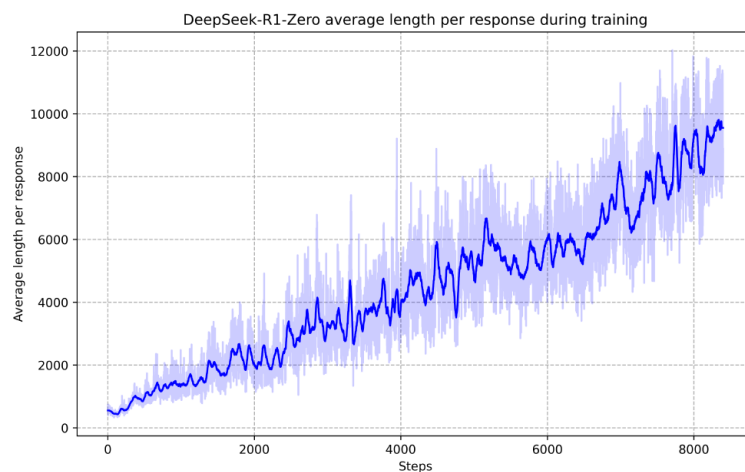


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

Likely the prompt template containing “The assistant first thinks about the reasoning process in the mind and then provides the user with the answer” helps with that.

Actually, the authors tried both Process Reward Models and MCTS, but those attempts didn't succeed.

Moreover, during RL the LLM somehow learns to use quite human-like patterns of thought:

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ... $(\sqrt{a - \sqrt{a + x}})^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$... Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be ... We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation:

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

However, despite these capabilities, DeepSeek-R1-Zero struggles with challenges like poor readability and language mixing.

So, several improvements are introduced into the training of DeepSeek-R1:

- Improving the unstable cold start of RL by first fine tuning on several thousands of high-quality data created with CoT and refined by human annotators.
- Post-RL SFT on both reasoning and non-reasoning (such as writing, factual QA, self-cognition, and translation) data. Partially, this stage tries to address the known problems that the model exhibits after RL.
- Additional RL for safety.

You can check some benchmark scores on the next page. I wouldn't say that we need to forget about all other models and only use DeepSeek from now on, but it's very competitive.

Benchmark (Metric)		Claude-3.5- Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
Architecture		-	-	MoE	-	-	MoE
# Activated Params		-	-	37B	-	-	37B
# Total Params		-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	84.0
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	92.3
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-	63.7

Table 4 | Comparison between DeepSeek-R1 and other representative models.

Imagine while Reasoning in Space: Multimodal Visualization-of-Thought

<https://arxiv.org/pdf/2501.07542>

Since the emergence of Visual Language Models such as GPT-4V, there have been discussions about visual Chains of Thoughts: reasoning processes, where text is interleaved with images. This sounds tempting indeed, especially because we, humans, don't just think in text (and Yann LeCun loves to remind about this). However, so far we're not flooded with papers about multimodal CoT; partially that's because we don't have too many Multimodal LLMs that could both process and generate text. And they are not cool at Visual CoT just yet. So, the authors decided to address this with fine tuning.

When training something like that, it's important to determine the right scope of visual tasks we consider. For example, doing some 2d geometry would be tempting, but VLMs are not very cool at working with intersecting lines and circles from my experience. The authors choose a set of less challenging tasks like navigating a maze:

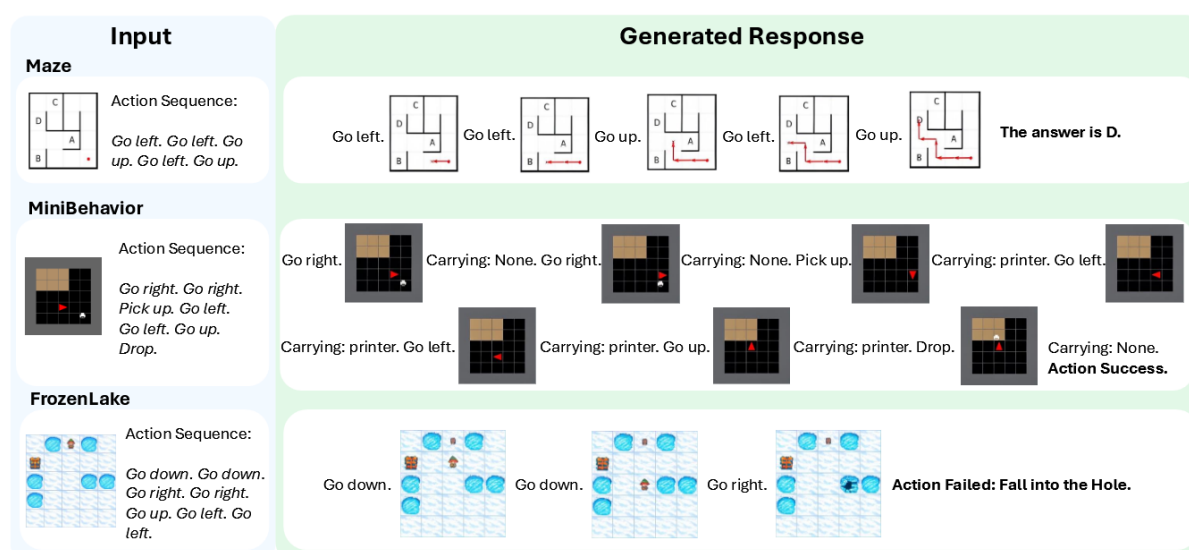


Figure 2: Illustrations of MVoT reasoning process. The interleaved verbal thoughts and visual thoughts are generated by MLLM seamlessly.

I want to underline that navigating a maze, for example, really benefits from visualization of movements; indeed, even I would trace the maze with my gaze while navigating it (or even with my finger:)).

The authors fine tune Anole-7B, one of the fine tunes of [Chameleon](#) by Meta, which, in turn, was one of the first Visual LMs that was able to both process and output images. In

comparison with the previous two papers that had struggles with creating training data, in this one data generation turns out to be reasonably easy due to existence of programmed simulators for all the tasks.

The authors compare several approaches:

- 1) Direct Prompting, without additional fine tuning and intermediate reasoning.
- 2) Pure-text Chain-of-Thought: The model is instruction-tuned to reason step-by-step, incorporating coordinates and environment layout described in text, before concluding with the final answer.
- 3) Training with Interleaved Text-Image Pairs (Interleaved), but the loss is only calculated for text tokens.
- 4) **Multimodal Visualization-of-Thought (MVoT)** – their main approach. The fine tuning loss has two components:

- Cross-entropy loss over both text and image output tokens. This makes sense because Chameleon’s visual encoder and decoder both work with a discrete vocabulary of visual tokens (think of VQ-VAE).
- **Token Discrepancy Loss** specifically aligns visual token embeddings with the ground truth embeddings, establishing better generation.

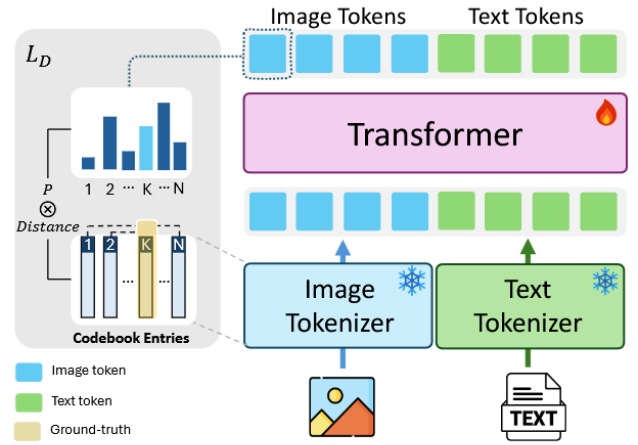


Figure 3: Illustration of MVoT training with token discrepancy loss $\mathcal{L}_D$.

Here are the results:

Table 2: Experimental results from different system variants across all tasks. *Coord* denotes whether the system is instructed to use textual coordinates during inference; *Layout* denotes whether the system first caption the environment with text for further inference; *Image* refers to whether the images of intermediate states are visible to the model during training. The **best** and **second best** performance are illustrated with corresponding colors. Methods with underlines are fine-tuned on our datasets. ↓ indicates worse performance than *Direct* baseline.

Model	Method	Training Variants			Output	Task		
		<i>Coord</i>	<i>Layout</i>	<i>Image</i>		MAZE	MINIBEHAVIOR	FROZENLAKE
GPT-4o	Zero-Shot Direct	-	-	-	Text	0.7100	0.4576	0.4976
	Zero-Shot CoT	-	-	-	Text	0.7386	0.4676	0.4664
	With Visual Thought*	-	-	-	Text	0.8556	0.6440	0.8021
Anole 7B	<u>Direct</u>	✗	✗	✗	Text	0.7171	0.7250	0.7788
	<u>CoT</u>	✓	✓	✗	Text	0.9792	0.9812	0.6148 ↓
	- w/o layout	✓	✗	✗	Text	0.7023 ↓	0.6000 ↓	0.5974 ↓
	<u>Interleaved</u>	✗	✗	✓	Text	0.8678	0.8406	0.6460 ↓
	<u>MVoT</u>	✗	✗	✓	Text, Image	0.9295	0.9514	0.8560

* Visual thought is generated by Anole 7B implemented with MVoT.

One may observe that for different tasks text-only CoT and MVoT switch places in this rating. I would say that for some tasks visual information may be described quite well in text, while for others textual description will be somewhat lacking or even confusing. This reminds me of the [Mind your Step \(by Step\)](#) paper which observed a curious pattern in Facial Recognition with VLMs: adding textual CoT for it harmed quality as opposed to zero-shot answering.

Titans: Learning to Memorize at Test Time

<https://arxiv.org/pdf/2501.00663>

The quest of making transformers fitting for long context has a long and interesting story. The issues that we need to overcome in this quest are of two types:

1. Efficiency: both self-attention's complexity and key-value cache's size are quadratic in sequence length.
2. Accuracy: despite the claims that LLMs are efficient with long context, that's not entirely true, and they tend to forget or distort important information even for medium-length context.

Also, these two considerations come with a trade-off: the more we try to optimize complexity and reduce memory usage, the more we harm accuracy.

Several remedies were suggested for tackling with long context, such as:

- Sliding window attention, which solves the problems of quadratic attention and KV-cache size) + some kind of additional global attention (for example, on some of the first tokens)
- Splitting the sequence into chunks; establishing full attention inside every chunk + inventing some memory summarization mechanism for each chunk and a way of translating the memory from one chunk to the next one.

One of the interesting recent examples was Star Attention

(<https://arxiv.org/pdf/2411.17116>).

Strictly speaking, Google didn't invent a totally new paradigm in their Titans paper; their solution boils down to the above parts. And the test-time training they relate to is just another recurrent memory mechanism. But still, they presented an interesting interpretation which is worth discussing. So, let's do this now!

To start with, the authors discuss the concept of memory and they relate it to **surprise**. We memorize whatever surprises us, and we're surprised by the changes in the environment. With formulas, it looks like:

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t),$$

where the gradient is the **surprise** and ℓ is some loss function. Next, the authors decide to distinguish between the role of the **past surprise** S_{t-1} and the **momentary surprise** $\nabla \ell$. These they relate through a formula that looks suspiciously like gradient descent with momentum:

$$\begin{aligned}\mathcal{M}_t &= \mathcal{M}_{t-1} + S_t \\ S_t &= \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t)\end{aligned}$$

Finally, whenever we have memory, we also need a forgetting mechanism, and to get one, we may introduce $\alpha_t \in [0,1]$:

$$\begin{aligned}\mathcal{M}_t &= (1 - \alpha_t) \mathcal{M}_{t-1} + S_t \\ S_t &= \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t)\end{aligned}$$

Now we need to define the loss ℓ . The authors do it as follows:

$$\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(k_t) - v_t\|_2^2,$$

where $k_t = x_t W_K$ and $v_t = x_t W_V$ are the key and the value from the self-attention mechanism, and $\mathcal{M}_{t-1}(\dots)$ is the memory represented by a neural network. In turn, extraction of the information from the memory will be done in the following way:

$$h_t = \mathcal{M}_t(q_t),$$

But... why these loss and extraction formula? These formulas are not easy to wrap your mind around, but I'll try to share whatever motivation I could come up with. Let $\mathcal{M}_{t-1}(k_t)$ be just one linear layer; given that k_t is a row, we'll have then $\mathcal{M}_{t-1}(k_t) = k_t \cdot \mathcal{M}_{t-1}$, where $\mathcal{M}_{t-1}$ is some matrix. For this case we can explicitly calculate the gradient (I've omitted $1 - \alpha_t$ for simplicity):

$$\begin{aligned}\mathcal{M}_t &= \mathcal{M}_{t-1} + S_t \\ S_t &= \eta_t S_{t-1} - \theta_t (2k_t^T k_t \mathcal{M}_{t-1} - 2k_t^T v)\end{aligned}$$

Substituting the second equation into the first one, we get:

$$\mathcal{M}_t = (1 - 2\theta_t k_t^T k_t) \mathcal{M}_{t-1} + 2\theta_t k_t^T v_t + \eta_t S_{t-1}$$

Now,

$$h_t = q_t \mathcal{M}_t = 2\theta_t q_t k_t^T v_t + q_t [(1 - 2\theta_t k_t^T k_t) \mathcal{M}_{t-1} + \eta_t S_{t-1}]$$

The summands $\mathcal{M}_{t-1}$ and S_{t-1} will, in turn, contain $k_{t-1}^T v_{t-1}$, $\mathcal{M}_{t-2}$ and S_{t-2} and so on. In the end, we'll get something like

$$\begin{aligned}h_t &= smth \cdot q_t k_t^T v_t + smth \cdot q_t k_{t-1}^T v_{t-1} + smth \cdot q_t k_{t-2}^T v_{t-2} + \dots = \\ &smth \cdot (q_t k_t^T) v_t + smth \cdot (q_t k_{t-1}^T) v_{t-1} + smth \cdot (q_t k_{t-2}^T) v_{t-2} + \dots =\end{aligned}$$

Now you can see that these terms are attention scores, just without softmax!

I'm not sure that this really validates the design choice, but this gives some intuition about what's happening here.

How to incorporate this kind of memory into a transformer model? The authors consider several options; let's consider two of them.

Option 1 (“Memory as a context”). In this interpretation, the authors split the long context into shorter segments $S^{(1)}, \dots, S^{(t)}, \dots$ and establish a recurrent process introducing the previous-segment memory into the current one.

Here is how the renewed self-attention will work how (see also picture on the next page):

- For the new segment, let's create the global query matrix: $Q = S^{(t)}W_Q$ and let's extract information from the previous memory: $h_t = \mathcal{M}_{t-1}(Q)$ [if $\mathcal{M}_{t-1}(\dots)$ is just one linear layer, this is $Q\mathcal{M}_{t-1}$].
- This information is concatenated to $S^{(t)}$ and task-related (and not context-related) “**persistent memory**” to get an updated chunk of embeddings:

$$\tilde{S}^{(t)} = \text{concat}(\text{persistent memory}, h_t, S^{(t)})$$
- We now run the self-attention: $y_t = \text{self_attn}(\tilde{S}^{(t)})$
- Then we update $\mathcal{M}_{t-1} \rightarrow \mathcal{M}_t$ with our “momentum” procedure
- Finally, we set the output of the self-attention layer as $o_t = y_t \otimes \mathcal{M}_t(y_t)$, where $\otimes$ is the elementwise product. This is something like gating: we use memory to understand which parts of information contained in y_t to push to the next layer.

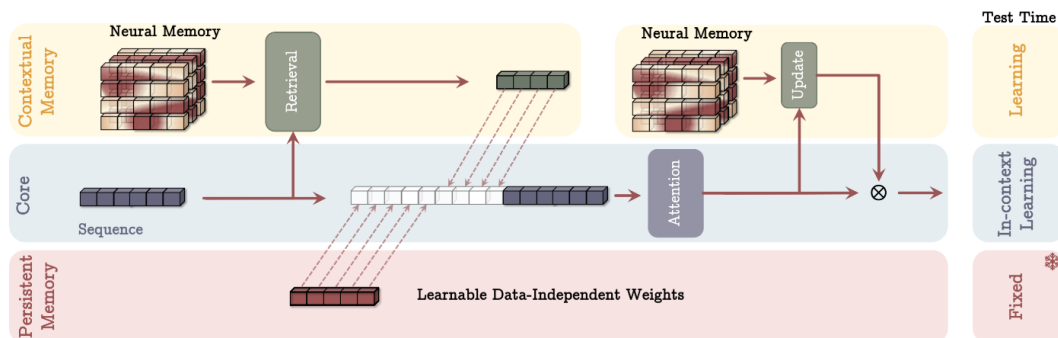
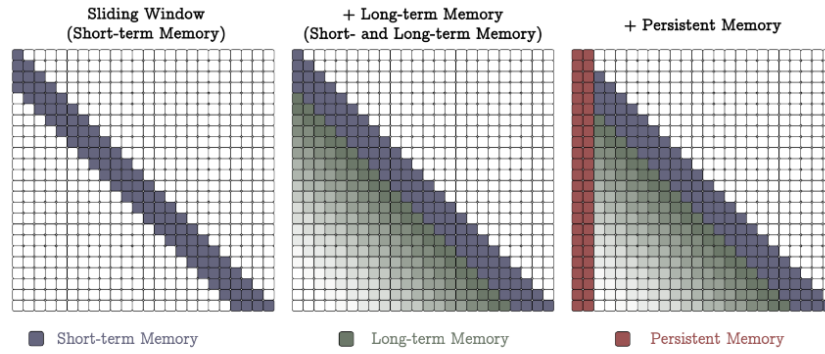


Figure 2: **Memory as a Context (MAC) Architecture.** This architecture includes three branches of (1) core, (2) contextual (long-term) memory, and (3) persistent memory. The core branch concatenates the *corresponding* long-term and persistent memories with the input sequence. Next, attention performs on the sequence and decides what part of the information should store in the long-term memory. At the test time, parameters corresponds to contextual memory are still learning, parameters corresponds to the core branch are responsible for in-context learning, and parameters of persistent memory are responsible to store the knowledge about tasks and so are fixed.

Note. Persistent memory vector is the same for the whole text; it may also change between different tasks.

Option 2 (“Memory as a gate”). In this approach, the authors use sliding window attention with additional recurrent memory. Note that the whole text isn't partitioned into segments.



(b) **Memory as Gating (MAG).** We use sliding window attention (SWA) as a short-term memory and our neural memory module as a long-term memory, combining by a gating.

How it works when processing a prompt:

- Given an input x , concatenate the persistent memory to it obtaining $\tilde{x} = \text{concat}(\text{persistent memory}, x)$, Note that x is the whole text here
- Run this through the attention $y = \text{Self_attention}(\tilde{x})$. Note that actually two attention mechanisms are employed here: the sliding window attention & the persistent attention on the persistent memory. This can be done in one pass using masked self-attention.
- Now, we need to run along the sequence ($t = 1, 2, \dots$), updating $\mathcal{M}_{t-1} \rightarrow \mathcal{M}_t$ and computing $\mathcal{M}(\tilde{x}_t)$ at the same time.
- Finally, use gating to produce the answer: $o = y \otimes \mathcal{M}(\tilde{x})$

During token-by-token autoregressive generation, similar things happen, but one token at a time:

- Given x_t , append it to the previous $\tilde{x}$,
- Get $y_t = \text{Self_attention}(\tilde{x})_t$, using KV-cache to avoid recalculating previous keys and values. This stays efficient, because only global persistent + sliding window attention is performed.
- Update $\mathcal{M}_{t-1} \rightarrow \mathcal{M}_t$ and compute $\mathcal{M}(\tilde{x}_t)$.
- Finally, use gating to produce the answer: $o_t = y_t \otimes \mathcal{M}(x_t)$

See also the picture:

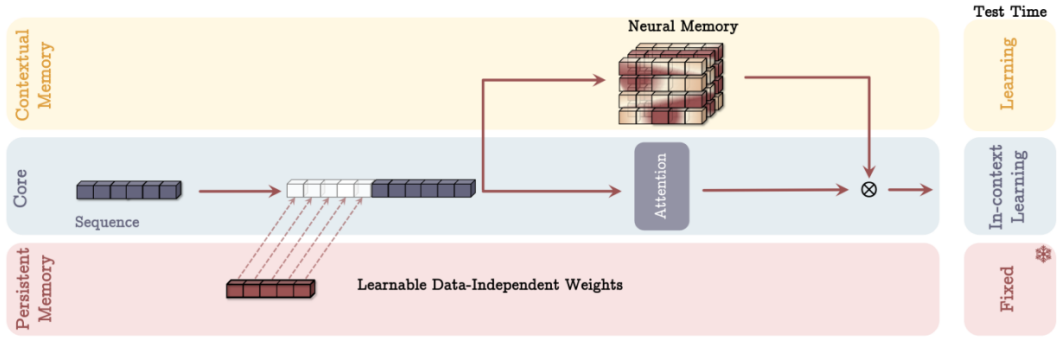


Figure 4: **Memory as a Gate (MAG) Architecture.** This architecture, similarly, has the three branches of (1) core, (2) contextual memory, and (3) persistent memory. It, however, incorporates only persistent memory into the context and combine memory with the core branch using a gating mechanism. At test time, the behavior is the same as Figure 2.

The paper presents some numerical results, but to tell the truth, they are not very convincing, because they only compare their model with quite weak and small LLMs. But in any case, this is quite an interesting technique.

Transformer²: Self-Adaptive LLMs

Imagine that you have many particular tasks you want your LLM to adapt to, and for each of them you don't have too much data. What should you do? Train a different LoRA for each of the tasks? Or is there a more flexible choice? The authors of Transformers² suggest such an approach.

For a pre-trained transformer, they suggest taking weight matrices W and calculating their [SVD \(Singular Value Decomposition\)](#). Let me briefly remind you what it is. SVD is the decomposition

$$W = U\Sigma V^T$$

where U and V are orthogonal and

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots)$$

is a (rectangular) diagonal matrix with

$$\sigma_1 \geq \sigma_2 \geq \dots \geq 0$$

Now, the idea will be to augment this with a diagonal matrix Z

$$W = U\Sigma ZV^T$$

and to fine tune this Z . The benefit is quite straightforward: during fine tuning, instead of training an $d_{in} \times d_{out}$ matrix W , we need to train only a diagonal matrix Z , that is a vector of length $\min(d_{in}, d_{out})$. Moreover, by introducing Z this way, we don't change

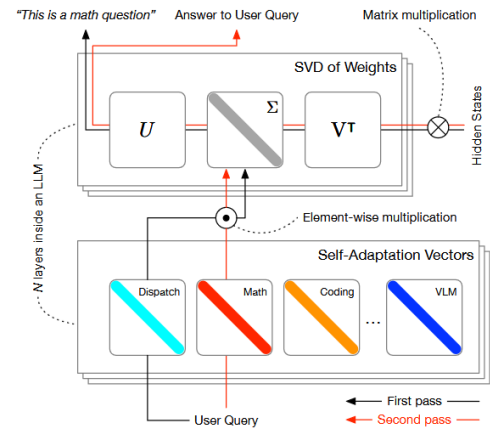


Figure 1: **Overview of Transformer².** In the training phase, we tune the scales of the singular values of the weight matrices to generate a set of “expert” vectors, each of which specializes in one type of tasks. In the inference phase, a two-pass process is adopted where the first applies the task-specific expert and the second generates the answer.

principal components in the SVD (see wiki for details), only their magnitude. This, in a sense, gives additional regularization.

Now, the training of these Z (there are many; potentially, for each matrix inside the transformer) surprisingly resembles what we've seen in DeepSeek-R1. It's done with RL. The reward $r(\hat{y}_i, y_i)$ is ± 1 , depending on the correctness of the answer. REINFORCE is used for optimization, and also, inspired by RLHF, the authors add the Kullback-Leibler penalty to avoid the LLM outputs changing too much. The final objective is

$$J(\theta_z) = \mathbb{E} [\log (\pi_{\theta_{W'}}, (\hat{y}_i | x_i)) r(\hat{y}_i, y_i)] - \lambda D_{\text{KL}}(\pi_{\theta_{W'}} \| \pi_{\theta_W}),$$

As always, $\pi_{\theta_{W'}}(\hat{y}_i | x_i)$ is the probability distribution of completions $\hat{y}_i$ based on a prompt x_i , and π_{θ_W} is the frozen LLM.

In this way, we may teach several “expert vectors” Z , each for its own narrow task/domain. Now, an interesting question is: if we get a new narrow task/domain at inference, can we adapt for it without training its own Z ?

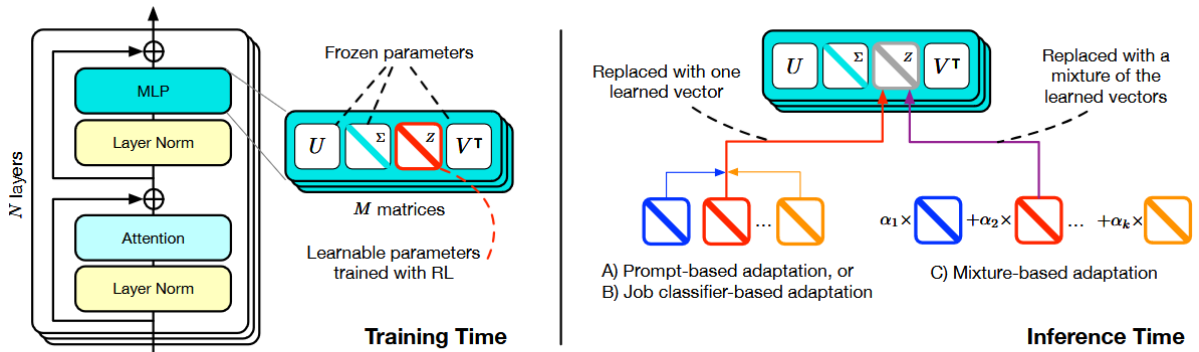


Figure 2: **Method overview.** Left) At training time, we employ SVF and RL to learn the “expert” vectors z ’s that scale the singular values of the weight matrices. Right) At inference time, we propose three distinct methods to adaptively select/combine the learned expert vectors.

The authors consider several adaptation techniques (see the right part of the schematic above).

Options A and B boil down to finding an existing “expert” optimally fitting for the new task. Option B is more interesting, and it requires a set of few-shot examples for the new task. Using these examples, the authors suggest fitting a new “expert” in the form of

$$\alpha_1 Z_{task1} + \dots + \alpha_k Z_{taskk}$$

The authors experiment with having only 10 few-shot examples. It’s too few for training, but seems to be ok for RL, at least for a simple cross-entropy method, which looks as follows:

0. Fix the initial distribution of $(\alpha_1, \dots, \alpha_k)$ (standard k -dimensional normal, for example)

1. Make 100 (or whatever) iterations of the following process:
 - a. Sample many $\alpha = (\alpha_1, \dots, \alpha_k)$ from the current distribution
 - b. Using each α , generate answers for the 10 few-shot examples, score them. To make scoring more reliable, you can generate multiple answers for each example.
 - c. Knowing these scores, take several best-performing α and fit a new random normal distribution on α that fits them
 - d. Repeat

(Not sure that the authors used normal distributions, maybe some other family, but that was a reasonable guess, I think.)

The results are shown on the next page, and they are quite nice.

Moreover, the authors check what happens if they graft the vectors Z trained on Llama to another model, Mistral. And even this gives some improvement. Remarkably though, when they shuffled SVD's principal direction before grafting, the quality degraded, which means that these directions are actually meaningful.

Table 2: **Self-adaptation on unseen tasks.** Normalized scores are in the parentheses.

Method	MATH	Humaneval	ARC-Challenge
LLAMA3-8B-INSTRUCT 3	24.54 (1.00)	60.98 (1.00)	80.63 (1.00)
+ LoRA	24.12 (0.98)	52.44 (0.86)	81.06 (1.01)
+ Transformer ² (Prompt)	25.22 (1.03)	61.59 (1.01)	81.74 (1.01)
+ Transformer ² (Cls-expert)	25.18 (1.03)	62.80 (1.03)	81.37 (1.01)
+ Transformer ² (Few-shot)	25.47 (1.04)	62.99 (1.03)	82.61 (1.02)
MISTRAL-7B-INSTRUCT-V0.3	13.02 (1.00)	43.29 (1.00)	71.76 (1.00)
+ LoRA	13.16 (1.01)	37.80 (0.87)	75.77 (1.06)
+ Transformer ² (Prompt)	11.86 (0.91)	43.90 (1.01)	72.35 (1.01)
+ Transformer ² (Cls-expert)	11.60 (0.89)	43.90 (1.01)	74.83 (1.04)
+ Transformer ² (Few-shot)	13.39 (1.03)	47.40 (1.09)	75.47 (1.05)
LLAMA3-70B-INSTRUCT	40.64 (1.00)	78.66 (1.00)	87.63 (1.00)
+ LoRA	25.40 (0.62)	73.78 (0.94)	83.70 (0.96)
+ Transformer ² (Prompt)	40.44 (1.00)	79.88 (1.02)	88.48 (1.01)

Table 5: **Cross-model z vector transfer.** Results from transferring the expert vectors trained on LLAMA3-8B-INSTRUCT to MISTRAL-7B-INSTRUCT-V0.3 with cross model few-shot adaptation.

Method	MATH	Humaneval	ARC-Challenge
<i>SVF training task</i>	<i>GSM8K</i>	<i>MBPP-pro</i>	<i>ARC-Easy</i>
MISTRAL-7B-INSTRUCT-V0.3	13.02 (1.00)	43.29 (1.00)	71.76 (1.00)
+ Llama SVF (ordered σ_i)	11.96 (0.92)	45.12 (1.04)	72.01 (1.00)
+ Llama SVF (shuffled σ_i)	10.52 (0.81)	40.24 (0.93)	70.82 (0.99)
+ Few-shot adaptation (cross-model)	12.65 (0.97)	46.75 (1.08)	75.64 (1.05)